Nawaf

Your Phase 1 work and preparation guide

1. Your responsibility

Your area: the custom list foundation and the complete rider lifecycle. You lead the code that stores riders, finds them, removes them, and keeps their ride records consistent.

You primarily implement	What you deliver
LinkedList<T> and Node<T>	Use/complete the classroom implementation and agree its exact API with the team. The ZIP does not contain this code. Node may already be nested inside LinkedList.
Person.java	Abstract shared person behavior: immutable ID, name, validated phone, ride history and toString.
Rider.java	Person subclass with email, home city and ID-based comparison.
RiderList.java	Every rider-list method, sorted ID storage, uniqueness and all-match removal.
RideSharingSystem.java	Your eight methods listed on page 4, plus the shared whole-ride cleanup helper.
Main.java	Options 1-4, integrated into Basam's common menu shell.

Your reviewer: Basam. **Integration tester for your area:** Waleed. You review Waleed's ride/scheduling code and test Basam's driver/time/loading code.

First actions:

1. Get the exact classroom LinkedList source. Agree on traversal, insertion, removal, size and cursor behavior before others depend on it.
2. Read IPerson, IRider and IRiderList. Keep every supplied interface unchanged.
3. Agree on constructors, list return behavior and history updates with Basam and Waleed.
4. Create compiling skeletons together, then implement your list and rider classes in the order on page 3.

Important ordering question: RiderList must remain sorted by ID, but assignment page 11 also asks for alphabetical listing in O(n). Confirm the display rule with the instructor. The agreed provisional design keeps a second custom name-ordered list of the same Rider references. Do not replace ID ordering with name ordering.

This guide follows the existing team split. It is a work checklist, not a claim that your code or tests are already finished. No schedule or effort estimates are included.

2. What you should study

You need a focused set of Java and linked-list skills. Learn each topic well enough to use it in the class or method shown below.

Must know	Where you use it
Classes, constructors, super, getters/setters	Person and Rider construction; private fields; public interface implementations.
Abstract classes, inheritance and interfaces	Person implements IPerson; Rider extends Person and implements IRider.
final and object references	Person ID never changes. Many different lists can reference the same Rider or Ride object.
Generics and null	LinkedList<T>, Node<T>, LinkedList<IRider>, LinkedList<IRide>, and empty-list boundaries.
Comparable and overriding	compareTo(IRider other) compares IDs. Do not add Comparable<Rider> or Comparable<IPerson>.
String equality and validation	Exact full-name/email/city matching; phone is a String containing exactly ten digits.
Exceptions	setPhoneNumber throws IllegalArgumentException on bad input; validate before assigning.
File reading, split and parseInt	Your rider CSV loader; five fields per row, no header.
Loops and return values	Traversal, search and deletion return data/status. Only Main does console input/output.

Your minimal linked-list checklist:

☐ Explain node data, next, head, and any classroom cursor; understand when a tail reference is useful.

☐ Traverse empty, one-node and larger lists safely.

☐ Insert before the head, between nodes and after the last node, preserving ID order.

☐ Find one ID/email and collect every matching name/city into another list.

☐ Delete first, middle, last and only nodes; remove every matching node without skipping adjacent matches.

☐ Maintain a correct count and build result lists using constant-time appends.

☐ Explain a shallow list copy: new nodes, same domain-object references.

Useful while building: the pattern [0-9]{10}, try-with-resources, assertions/debugging and small Git commits. A character loop can validate a phone instead of a regular expression.

Not required: doubly linked lists, circular lists, reversal, recursion, iterator implementation, advanced generics or streams. Java collection structures such as ArrayList, java.util.LinkedList and HashMap are prohibited.

Ready-to-start exercise: insert 20, 10, 30, 25 and obtain 10, 20, 25, 30. Then remove two adjacent matching records without losing or skipping another node.

3. Your class implementation checklist

Build order: classroom list contract -> list boundary tests -> Person -> Rider -> RiderList. Work against the supplied IRide interface; you do not need Waleed's complete Ride implementation to start.

Person implements IPerson:

- Store final int id, name, phoneNumber as String, and a custom LinkedList<IRide> history.
- Implement getId, getName, setName, getPhoneNumber, setPhoneNumber, getRideHistory and toString.
- Make Person abstract. Initialize an empty history. Reuse phone validation during construction.
- Reject null or non-ten-digit phones with IllegalArgumentException. Do not require a 05 prefix: the stated rule is ten digits only.
- Under the agreed design, getRideHistory returns the actual list for controlled system updates. A history copy would not work if the scheduler expects to update it.

Rider extends Person implements IRider: add email and homeCity; implement getEmail, setEmail, getHomeCity, setHomeCity and compareTo(IRider). Compare using Integer.compare(getId(), other.getId()). Format useful rider fields in toString without printing or recursively expanding ride histories.

RiderList method	What your implementation must do
add	Reject duplicate ID and duplicate email; insert by ID; update the agreed name index.
findById / findByEmail	Return the one matching rider, or null under the agreed miss policy.
findByName / findByHomeCity	Return every exact match in a new custom list; return an empty list on no matches.
getAll	Return an ID-ordered shallow snapshot under the agreed list contract.
removeById / removeByEmail	Remove at most one; return true/false; update both indexes.
removeByName / removeByHomeCity	Remove ALL matches and return the number removed. Use full-name equality, not a first-name or substring test.
size	Return a maintained count. If your actual implementation recounts nodes, its cost is O(n), not O(1).

Avoid these bugs: do not advance twice if classroom remove already moves the cursor. Do not use == to compare String contents. Do not rescan the result-list tail for every append. For bulk removal with two indexes, filter each index in one pass instead of performing a full second-index search for every removed record.

List-level deletion manages rider records. Your system-level removeRider method additionally manages rides and histories; do not hide the cascade inside an isolated RiderList that has no access to the other registries.

4. Your system methods and menu work

Implement these **eight existing IRideSharingSystem methods**. Preserve their supplied signatures; coordinate edits to the shared file with Basam and Waleed.

Method	Your responsibility
loadRidersFromCSV	Read every row, validate five fields, construct Rider, call addRider, and return loading status.
addRider	Enforce rider ID and email uniqueness; return false for normal invalid/duplicate requests.
searchRiderById	Delegate to the rider ID search; return the object or null.
searchRiderByEmail	Delegate to the unique-email search.
searchRidersByName	Return all exact full-name matches.
searchRidersByHomeCity	Return all exact home-city matches.
getAllRiders	Supply all riders; use the agreed display order and maintained name index if alphabetical display is required.
removeRider	Remove the rider and perform every required ride/history cascade.

Your CSV mapping: riderId, name, email, phoneNumber, homeCity. Parse ID as int, preserve phone as String, and pass fields in the team's agreed constructor order. The supplied file has **100 records and no header**. Use split(",", -1) and check field count; do not discard the first rider. Read files without console printing.

Shared loader policy: the current provisional agreement is fail-fast: false on the first bad row, with already accepted rows retained. The assignment does not specify rollback, so confirm this with the team/instructor. Main must not treat a partial load as complete success.

Main options: 1 -> getAllRiders; 2 -> searchRidersByHomeCity; 3 -> searchRidersByName; 4 -> construct Rider and call addRider. Print every returned element or a clear empty-results message. Catch construction/input errors in Main.

removeRider flow:

1. Find the rider; return false if absent. Snapshot their history so removal cannot break traversal.
2. Delete their private rides everywhere.
3. Remove them from each shared ride and from their own history. Keep a nonempty shared ride, including one with only one remaining rider.
4. If a shared ride becomes empty, delete it globally and from its driver's history. Finally remove the rider from all rider indexes.

You lead removeRideEverywhere(IRide): an internal helper that removes a whole ride from RideList, its driver's history and every participant's history. Basam uses it for driver removal; Waleed reviews its interaction with participant lists. Do not add it to the supplied interface.

5. Tests and Big-O you must deliver

Run these on fresh fixtures. Check returned values, list order, counts, and history membership, not only printed output.

Test	Expected result
Empty and single-node lists	No null error; size correct; deleting the only node leaves a valid empty list.
Insert 20, 10, 30, 25	Stored ID order is 10, 20, 25, 30.
Duplicate ID or email	Reject; no count/index/history change. Repeated names remain allowed.
Name/city search	Every matching rider returned; an absent value returns an empty custom list.
Head/middle/tail deletion	Correct links and count; deleting a missing ID returns false.
Consecutive/all-match deletion	No skipped records; return exact removal count; both indexes agree.
Invalid phone setter	Throws IllegalArgumentException; previous valid value remains unchanged.
Result-list isolation	Removing a result node does not remove a stored rider.
Remove rider's private ride	Global ride and driver's history entry disappear.
Shared ride: 2 -> 1 -> 0 riders	First removal keeps the ride; last removal deletes it and its driver history entry.
No rides / absent rider	Existing rider removes successfully; absent rider returns false without changes.
Booking after cascade	Deleted rides cannot cause a stale-history scheduling conflict.

Supplied-data checks: search Mohammed Al-Otaibi returns IDs 10 and 55. RiyadhNorth returns IDs 18, 36, 52, 55, 61 and 100. A clean rider load contains 100 riders.

Your analysis	Expected bound under the agreed design
List traversal/search/sorted insertion	O(n): at most n visits, then constant reference changes.
Rider searches, add, copies and removals	O(r), including a constant number of index passes; result append must be O(1).
Stored size; ID comparison	O(1).
Loading r riders into an empty sorted registry	O(r^2): 0 + 1 + ... + (r-1) node visits in the worst case.
removeRider using repeated linear cleanup	Safe upper bound O(r + m^2 + P); do not call every cascade O(r).

Here r is riders, m is rides and P is total rider memberships across rides. A whole-ride deletion may scan RideList plus the relevant histories. Derive the actual helper and cascade cost from your loops. Add getters, setters, constructors and toString to your final method analysis; output formatting costs depend on output length.

If the classroom list cannot append in O(1), change the implementation approach or the complexity claim. Do not claim O(n) copying while repeatedly scanning the growing result list.

6. Team handoffs and your completion checklist

Teammate	What you exchange
Basam	You supply Person, the agreed list API, rider loading/search and whole-ride cleanup. Receive immutable DateTime behavior, driver lookup/history behavior and the Main shell. Basam reviews your code; you test his driver/time/load paths.
Waleed	You supply rider existence/search/history behavior and list operations. Receive private/shared rider access, participant removal and RideList removal. You review his ride/scheduling code; he tests your rider cascades and list boundaries.

Agree before merging: constructor signatures; list cursor semantics; null versus empty-list returns; history ownership; exact string matching; email uniqueness; name-index policy; and how stored-object changes preserve indexes. Rider ID 1 and driver ID 1 may coexist because their uniqueness is checked in separate registries.

Whole-project facts you must explain: every accepted ride uses the same ride reference globally and in the driver's and each rider's history; the overlap rule uses two strict comparisons joined by AND; people load before rides; only Main performs console I/O; a removed driver causes all assigned rides to be deleted.

Demo practice:

- Trace sorted insertion and deletion of two adjacent matching nodes.
- Explain why duplicate email is rejected but duplicate full name is allowed.
- Trace removing a rider from a private ride and from a shared ride that still has another participant.
- Explain why a new ride starting exactly when the old one ends is allowed.
- Navigate another member's menu option and identify the actual system method.

You are finished when:

☐ All your classes compile with the unchanged supplied interfaces and the other members' classes.

☐ Every assigned method and Main option works and has meaningful tests.

☐ Both rider indexes, counts and histories remain consistent after every operation.

☐ Basam has reviewed your work and Waleed has tested it; you have reviewed/tested their assigned areas too.

☐ Your report section derives worst-case complexities from the final code, including helper calls.

☐ Your actual coding, review, testing and integration contributions are recorded.

☐ You can explain the entire program, including scheduling and driver deletion.

☐ You help verify the final source ZIP, PDF report with actual student IDs, and required demonstration.

Do not modify: supplied interfaces or VehicleType. **Do not use:** Java collection structures. **Confirm with the instructor:** alphabetical rider/driver display, the missing classroom list API, and repeated-pickup participant lookup. Clarification work is shared, not yours alone.

Sources: current CSC212_Phase1_Team_Roadmap.pdf, especially Sections 6-8 and 14; supplied RideSharing_Assignment.pdf pp. 1-14; IPerson, IRider, IRiderList, IRideSharingSystem; riders_100.csv. This personal guide preserves the agreed team allocation.